

Weird proxies/2 and a bit of magic (English translation)

Weird proxies/2 and a bit of magic - @speakerdeck, GreenDog, Speaker Deck.

- Published: 2021-09-01
- Original: <<https://speakerdeck.com/greendog/2-and-a-bit-of-magic>>
- Preserved from: <https://speakerdeck.com/greendog/2-and-a-bit-of-magic> (live) on 2026-08-09
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content (translated into English)

Machine translation of [2021-speaker-deck-weird-proxies-2-bit-magic.md](#), which holds the source's own words. Code, payloads, type names, URLs and CVE identifiers were masked before translating and restored after, so they are byte-identical to the original.

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Weird proxies/2 and a bit of magic - Speaker Deck

Weird proxies/2 and a bit of magic

<https://zeronights.ru/en/reports-en/weird-proxies-2-and-a-bit-of-magic/>

Reverse proxies and their variations are used everywhere in modern web applications for routing, caching, and access differentiation. This talk is dedicated to new research results about different reverse proxies and new possibilities brought by HTTP/2. It is a collection of tricks for exploiting various misconfigurations.

Results - https://github.com/GrrrDog/weird_proxies

[image: Avatar for GreenDog]

GreenDog

September 01, 2021

More Decks by GreenDog

[See All by GreenDog](#)

[

How to break SAML if I have paws?

](https://speakerdeck.com/greendog/how-to-break-saml-if-i-have-paws)

[ greendog](https://speakerdeck.com/greendog)

1

3k

[

Reverse proxies & Inconsistency

](https://speakerdeck.com/greendog/reverse-proxies-and-inconsistency)

[ greendog](https://speakerdeck.com/greendog)

3

5.9k

[

MITM Attacks on HTTPS: Another Perspective

](https://speakerdeck.com/greendog/mitm-attacks-on-https-another-perspective)

[ greendog](https://speakerdeck.com/greendog)

2

890

[

Deserialization vulnerabilities

](https://speakerdeck.com/greendog/deserialization-vulnerabilities)

[ greendog](https://speakerdeck.com/greendog)

1

1.1k

Other Decks in Programming

[See All in Programming](#)

[

JAWS-UG Yokohama #102 AWS End-of-Service Memorial LT Meetup: AWS Services That Can't Rest in Peace — Today, We Memorialize Three of Them —

](https://speakerdeck.com/maroon1st/jaws-ugheng-bang-number-102-awssazhong-gong-yang-lthui-cheng-fo-dekinai-aws-sabisutati-ben-ri-san-ti-gong-yang-simasu)

[ maroon1st](https://speakerdeck.com/maroon1st)

0

320

[

Are Those Savings Actually Saving You Yen?

](https://speakerdeck.com/isamumumu/sonojie-yue-yuan-ninatutemasuka)

[[\[image: Avatar for isamu asakawa\]](#) isamumumu](https://speakerdeck.com/isamumumu)

1

690

[

Introduction to Web Application Tuning with Laravel / web_application_tuning_101

](https://speakerdeck.com/hanhan1978/web-application-tuning-101)

[[\[image: Avatar for Ryo Tomidokoro\]](#) hanhan1978](https://speakerdeck.com/hanhan1978)

4

1.7k

[

Turning Siloed Knowledge into a Map AI Can Follow

](https://speakerdeck.com/pkshadeck/supporterz-colab-lt-ai-festival-20260728)

[[\[image: Avatar for PKSHA Technology \(Parksha Technology\)\]](#) pkshadeck]

(https://speakerdeck.com/pkshadeck)

[PRO](#)

1

140

[

PHP Beginner Session 2026 — Understanding What Generative AI Can't Show You: Learning How Things Work Through LAMP, Now More Than Ever —

](https://speakerdeck.com/kashioka/phpchu-xin-zhe-setusiyon2026-sheng-cheng-aidehajian-enaili-ce-wozhi-ru-jin-dakaralampwotong-siteshi-zu-miwoxue-bu)

[[\[image: Avatar for Hideo Kashioka\]](#) kashioka](https://speakerdeck.com/kashioka)

0

840

[

From Vibe Coding to Agentic Workflow

](https://speakerdeck.com/starfish719/vibecodingkaraagenticworkflowhe)

[[\[image: Avatar for starfish719\]](#) starfish719](https://speakerdeck.com/starfish719)

0

260

[
Detecting Compromised CI with eBPF and Cilium Tetragon
](https://speakerdeck.com/lizrice/detecting-compromised-ci-with-ebpf-and-cilium-tetragon)
[ lizrice](https://speakerdeck.com/lizrice)

0
150

[
What's New in Android 2026
](https://speakerdeck.com/veronikapj/whats-new-in-android-2026)
[ veronikapj](https://speakerdeck.com/veronikapj)

0
250

[
Built Our Own Background Agent at LayerX
](https://speakerdeck.com/layerx/built-our-own-background-agent-at-layerx-number-aidevex-findy)
[ layerx](https://speakerdeck.com/layerx)

PRO

9
4.9k

[
The Story of Building the jsmini JavaScript Engine
](https://speakerdeck.com/yosuke_furukawa/jsmini-javascript-engine-wozuo-tutemitahua)
[ yosuke_furukawa]
(https://speakerdeck.com/yosuke_furukawa)

PRO

0
290

[
Engineering “Other Than Writing Code” — Four Tips for Driving a Billing Platform Migration Project
](https://speakerdeck.com/yuriko1211/engineering-beyond-coding)
[.png yuriko1211](https://speakerdeck.com/yuriko1211)

0
580

[
A “Second Brain” Built with Claude Code That “Keeps Work Moving Even While You Sleep”
(https://speakerdeck.com/tomoyafujita2016/qin-tetemoshi-shi-gajin-mu-claude-codedezu-mudier-nonoy)
[[image: Avatar for t-fujita24](#) tomoyafujita2016](https://speakerdeck.com/tomoyafujita2016)
0
300

Featured

[See All Featured](#)

[
Avoiding the “Bad Training, Faster” Trap in the Age of AI
(https://speakerdeck.com/tmiket/avoiding-the-bad-training-faster-trap-in-the-age-of-ai)
[[image: Avatar for Mike Taylor](#) tmiket](https://speakerdeck.com/tmiket)
0
200
[
A designer walks into a library...
(https://speakerdeck.com/pauljervisheath/a-designer-walks-into-a-library-dot-dot-dot)
[[image: Avatar for Paul Jervis Heath](#) pauljervisheath](https://speakerdeck.com/pauljervisheath)
211
24k
[
Optimising Largest Contentful Paint
(https://speakerdeck.com/csswizardry/optimising-largest-contentful-paint)
[[image: Avatar for Harry Roberts](#) csswizardry](https://speakerdeck.com/csswizardry)
37
3.9k
[
Google's AI Overviews - The New Search
(https://speakerdeck.com/badams/googles-ai-overviews-the-new-search)
[[image: Avatar for Barry Adams](#) badams](https://speakerdeck.com/badams)
0
1.1k

[
Information Architects: The Missing Link in Design Systems
](https://speakerdeck.com/soysaucechin/information-architects-the-missing-link-in-design-systems)

[[\[image: Avatar for Michelle Chin\]](#) soysaucechin](https://speakerdeck.com/soysaucechin)

0

1k

[
Digital Projects Gone Horribly Wrong (And the UX Pros Who Still Save the Day) - Dean Schuster
](https://speakerdeck.com/uxyall/digital-projects-gone-horribly-wrong-and-the-ux-pros-who-still-save-the-day-dean-schuster)

[[\[image: Avatar for UX Y'all\]](#) uxyall](https://speakerdeck.com/uxyall)

1

2.2k

[
The innovator's Mindset - Leading Through an Era of Exponential Change - McGill University 2025
](https://speakerdeck.com/jdejongh/the-innovators-mindset-leading-through-an-era-of-exponential-change-mcgill-university-2025)

[[\[image: Avatar for Jean-Marc De Jonghe\]](#) jdejongh](https://speakerdeck.com/jdejongh)

PRO

1

230

[
Code Reviewing Like a Champion
](https://speakerdeck.com/maltzj/code-reviewing-like-a-champion)

[[\[image: Avatar for maltzj\]](#) maltzj](https://speakerdeck.com/maltzj)

528

40k

[
GitHub's CSS Performance
](https://speakerdeck.com/jonrohan/githubs-css-performance)

[[\[image: Avatar for Jon Rohan\]](#) jonrohan](https://speakerdeck.com/jonrohan)

1033

470k

[
SEO for Brand Visibility & Recognition
(<https://speakerdeck.com/aleyda/seo-for-brand-visibility-and-recognition>)
[[image: Avatar for Aleyda Solis](#) aleyda](<https://speakerdeck.com/aleyda>)
0
4.7k
[
The Invisible Side of Design
(<https://speakerdeck.com/smashingmag/the-invisible-side-of-design>)
[[image: Avatar for Vitaly Friedman](#) smashingmag](<https://speakerdeck.com/smashingmag>)
301
52k
[
Winning Ecommerce Organic Search in an AI Era - #searchnstuff2025
(<https://speakerdeck.com/aleyda/winning-ecommerce-organic-search-in-an-ai-era-number-searchnstuff2025>)
[[image: Avatar for Aleyda Solis](#) aleyda](<https://speakerdeck.com/aleyda>)
1
2.1k

Transcript

-

[Weird proxies/2](#)

-

[About me - Security researcher at Invicti - Web security](#)

enthusiast / pentester <https://github.com/GrrrDog> <https://twitter.com/antyurin> - Co-organizer of Defcon Russia 7812 <https://t.me/DCG7812>

-

[Weird proxies/2 - ZeroNights 2018 - "Reverse proxies & Inconsistency"](#)

- https://github.com/GrrrDog/weird_proxies - Research Collisions - Thanks to contributors

-

Reverse proxy - Front-End - Reverse proxy/Load balancer/Cache proxy/... -

Back-end - Origin/Web-Server/...

-

Front-end Request - Route to back-end - Route to endpoints

- Rewrite path/query - Deny access - Headers modification - ... Response - Cache - Headers modification - Body modification - ...

-

HTTP/1.1 Request-line: method SP request-target SP HTTP-version CRLF GET /path/

HTTP/1.1 - Request-line Host: target.com Header: value

-

Server - Receive Request - Parse - Normalize (/path/.. ->

/path/, urldecode) - Apply rules (deny /admin, proxy to /endpoint2/) - Recreate Request (urlencode, initial/norm. path) - Send Request Inconsistency between Front-end and Back-end

-

Host misrouting Haproxy: - Doesn't support Absolute URI - Forwards

as is GET http://backend.com/q?name=X&type=Y HTTP/1.1 Host: target.com Nginx: - backend.com "rewrites" Host header

-

Host misrouting GET http://localhost/q?name=X&type=Y HTTP/1.1 Host: target.com Haproxy: - target.com

Nginx: - localhost

-

Nginx - Accepts raw bytes (0x01-0x20) in path as-is GET

/path/<TAB>HTTP/1.1 HTTP/1.1 Path => /path/<TAB>HTTP/1.1

-

Nginx - No trailing slash in proxy_pass proxy_pass http://backend -

Forwards the initial path After: GET /path/<TAB>HTTP/1.1 HTTP/1.1

-

Gunicorn - Reads until 1st whitespace with HTTP/1.1 - Accepts

arbitrary string in HTTP version GET /path/ HTTP/1.1 anything

-

Path misrouting Nginx + Gunicorn location /public/path { proxy_pass http://backend_gunicorn;

}

-

Path misrouting GET /admin/<TAB>HTTP/1.1/../../public/path HTTP/1.1 Nginx: - After normalization -

/public/path - Forwards - /admin/<TAB>HTTP/1.1/../../public/path Gunicorn: - After parsing - /admin/

-

Caddy - urldecodes, but doesn't normalize the path - Bypasses,

misrouting - //admin/ ../admin/ /Admin/ - Support fastcgi - php-fpm - as "back-end" - Idea: path traversal in SCRIPT_FILENAME for php-fpm

-

Caddy @phpFiles path *.php reverse_proxy @phpFiles 192.168.78.111:9000 { transport fastcgi

{ split .php } }

-

Caddy - Caddy's fastcgi module internally normalizes path - <=2.4.?

- Path traversal vuln - /../../../../../any/path/www/index.php HTTP/1.1 - <https://github.com/caddyserver/caddy/pull/4207> - no cve :(

-

URL/Header manipulation Caddy: route /prefix/* { uri strip_prefix /prefix/ reverse_proxy

192.168.78.111:9999 } Before - GET /prefix/http://localhost/admin HTTP/1.1 After - GET http://localhost/admin HTTP/1.1

-

URL/Header manipulation - Nginx \$uri - normalized URI (urldecoded) -

Works for any normalized value location /uri { proxy_pass http://192.168.78.111:9999;
proxy_set_header X-uri \$uri; }

-

URL/Header manipulation - %0d%0a -> \r\n - Request Splitting GET

/uri/%0d%0a%0d%0aGET%20/admin%20HTTP/1.1%0d%0aHost:localhost%0d%0a%0d%0a
HTTP/1.1

-

URL/Header manipulation After Nginx, a web server sees 2 requests:

GET /uri/%0D%0A%0D%0AGET%20/admin%20HTTP/1.1%0D%0AHost:localhost%0D%0A%0D%0A
HTTP/1.0 Host: target.com X-uri: /uri/ GET /admin HTTP/1.1 Host:localhost

-

Deep rabbit hole - Send url encoded value location ~

/header/(.)? { proxy_pass http://192.168.78.111:9999/test/\$1; } - Send url decoded value (\r\n) location ~
/header/([^\r\n]+)/ { proxy_pass http://192.168.78.111:9999/test/\$1; }

-

Deep rabbit hole - Send url decoded values location ~

/header/([^\r\n]+)/ { proxy_pass http://192.168.78.111:9999/test/\$1; } - 0_o Thanks to
<https://labs.detectify.com/2021/02/18/middleware-middleware-everywhere-and-lots-of-misconfigurations-to-fix/>

-

Complex systems AWS, Fastly, CloudFlare, StackPath, etc (tested in 2019)0

- many components (routing, caching, WAF, etc) - inconsistency between internal components
Normalize (/path/.. / -> /path/, urlencode) Apply rules (deny /admin, proxy to /endpoint2/)
Recreate Request (urlencode, initial/norm. path)

-

Restriction bypass - Restricted access to /admin - Bypasses: //admin/

/Admin/ /%61dmin/ /aaa/.. /admin - + Path misrouting - AWS - Fastly - patched(?) documentation -
CloudFlare - <https://developers.cloudflare.com/rules/normalization>

-

Nginx? Errors? Examples AWS ELB - // -> / ,

but /// -> /// - Forwards spaces in path AWS CloudFront - Deletes spaces in path - Forwards the
initial path - If space in the path, forwards normalized path StackPath - Incorrectly normalizes /.. / in

some cases

-

API gateway - Egress proxy - Microservices - Routing -

Authentication - Add header to request

-

API gateway - Kong - Based on Nginx - Didn't

normalize path - /public/../admin -> /public/../admin - CVE-2021-27306

[https://sewan.medium.com/cve-2021-27306-access-an-authenticated-route-on-kong-a pi-gateway-6ae3d81968a3](https://sewan.medium.com/cve-2021-27306-access-an-authenticated-route-on-kong-a-pi-gateway-6ae3d81968a3)

-

API gateway - Traefik doesn't normalize path - Caddy doesn't

normalize path - Envoy depends on configuration - CVE-2019-9901 (<1.9.1) - Doesn't normalize path in older versions, by default(?) - Many options to setup normalization - Hard to configure properly

-

Header smuggling API Gateway checks auth and adds header -

e.g x-user: admin CGI* "-" is converted to "_" - Traefik, Caddy, Envoy* proxy them - Caddy proxies to fastcgi

-

Caddy: Underscore To Caddy: GET / HTTP/1.1 x_user: ADMIN After

Caddy* (fastcgi): GET / HTTP/1.1 x-user: anonymous x_user: ADMIN

-

Envoy: Double headers - name: envoy.filters.http.ext_authz typed_config: "@type": type.googleapis.com/envoy.extensions.filters.http.ext_authz.v3.ExtAuthz http_service:

server_uri: uri: ext_authz cluster: ext_authz-http-service timeout: 0.250s authorization_response: allowed_upstream_headers: patterns: - exact: x-user

-

Envoy: Double headers To Envoy GET / HTTP/1.1 x-user: asdasd

x-user: ADMIN - Tested on 1.14.4 After Envoy GET / HTTP/1.1 x-user: User_from_ext_auth x-user: ADMIN

Special symbols Test normalization of header names Traefik(1.7.7) - Accepts

header with a trailing space - Forwards without the trailing space - net/http - Before Go 1.13.1 and Go 1.12.10 (CVE-2019-16276)

Special symbols To Traefik GET / HTTP/1.1 x-user : ADMIN

After Traefik GET / HTTP/1.1 x-user: ADMIN x-user: anonymous

Web cache poisoning Header smuggling with multiple headers Nginx allows

multiple Host headers - Nginx uses 1st Host - fastcgi_pass or uwsgi_pass gets 2nd Host - App trusts Host header - Cache proxy forwards multiple Host headers (smuggled), 2nd Host header is injection point

Web cache poisoning Header smuggling and Range header - Range

header returns part of response body Request: - Range: bytes=33- Response: - Status - 206 - Content-Range: bytes 33-106/107

Range header - 206 is allowed to cache, by standard

- Most cache proxies don't cache, by default - Can be configured to cache - Varnish doesn't proxy Range header*

Range header sub vcl_fetch { # ... if (beresp.status >=

200 && beresp.status < 300) { set beresp.cacheable = true; } # ... }
<https://docs.fastly.com/en/guides/http-status-codes-cached-by-default>

Range header - Browser treats 206 as 200 - If

1 range, server returns same Content-Type - Browser renders part of response - Attack can control part of response - Back-end must support Range

Range header - Send request with smuggled Range header -

Cache proxy forwards it to Back-end - Back-end returns a part of response - Cache proxy caches the part - Victim's browser renders only the part - Attacker can escape context

-

HTTP/2 - Binary protocol - Length for fields - Frames

(Headers, Data) - High-level compatibility with HTTP/1.1 - Pseudo-headers - :method, :scheme, :authority, :path - HTTP/2 -> HTTP/1.1

-

HTTP/2 HTTP/1.1: GET /path/ HTTP/1.1 Host: target.com Header: value HTTP/2:

:method:GET :path:/path/ :authority:target.com :scheme:https header:value

-

Restriction bypass Before :method:GET :path:<SP>/admin/ :authority:target.com :scheme:https Header: value <sp>

- white space After GET<SP><SP>/admin/ HTTP/1.1 Host: target.com Header: value

-

- :authority - host header - absolute uri Collision

-

Host misrouting Before Haproxy :authority:target.com host:localhost After Haproxy GET /

HTTP/1.1 Host:localhost

-

Haproxy - Prepend :scheme to path - If it's not

http or https - Allows any symbols in :scheme - except \x00 \x0a \x0d - v2.4.0

-

Misrouting Before Haproxy :path:/any :scheme:http://localhost/admin? :authority:target.com After Haproxy GET http://localhost/admin?://target.com/any

HTTP/1.1

-

Envoy - Allows spec symbols and \x20 \x09 in :method

- v1.18.3 - The same for Haproxy

-

Misrouting Before Envoy :method:GET /admin? :path:/ After Envoy GET /admin?

/ HTTP/1.1 Nginx: /admin? /

-

CPDoS and HTTP/2 - Cache Poisoning DoS - When we

can poison cache proxy with “broken response” - Usually, when proxy caches 4xx, 5xx resps - Meta symbols in header names - Oversized headers

-

Conclusion - New web-servers - old problems - Lack of

path normalization - Configuration-dependent vulnerabilities - New protocol - new opportunities

-

Next steps - Reading list of related articles/presentations - Results

- Docker images https://github.com/GrrrDog/weird_proxies

-

Thank you Questions

Archived from <https://speakerdeck.com/greendog/2-and-a-bit-of-magic>. Rendered offline from the local Markdown copy.